# ADAPTIVE ONLINE TRAINING OF NEURAL NETWORK FOR CONTROL OF A QUADROTOR HELICOPTER

Project # 2 Report for ENEL 667 Intelligent Control

November 27, 2016

Jason Paul
951279

# TABLE OF CONTENTS

# 1 INTRODUCTION

In this second project of the course, the goal is to derive a nonlinear intelligent control with adaptive online learning. In the situation where nonlinear system dynamics have parameters that are either initially uncertain or may change with time, adaptive online learning can be a solution.

As for Project 1, the nonlinear system to be controlled is the quadrotor helicopter (quadrotor).

The scope of this project is limited to the stability and tracking ability of the quadrotor in a hovering mode when subjected to a trajectory input and disturbances. The chosen trajectory is an oscillation (sinusoid) of the quadrotor pitch. Disturbances include: (1) the removal of quadrotor mass, as in the case of a package delivery, and (2) simulated wind, applied as a sinusoidal torque oscillation about the roll ($\phi$) axis.

First the mathematical model of the system is introduced (same as for Project 1). The selected control law, weight update (adaptation) law, and neural network are then discussed and justified, followed by a proof of stability for the control method. The derived control is applied in closed loop simulation scenarios to illustrate general stability, disturbance rejection, trajectory tracking ability, and weights convergence. Sensitivity of weights convergence to values of the adaptation gain $\beta$ and robust weights update constant $\nu$ is also explored.

# 2 MATHEMATICAL MODEL

Figure 1 illustrates the Cartesian coordinate system and sign convention for quadrotor rotations and translation as well as rotor spin directions.

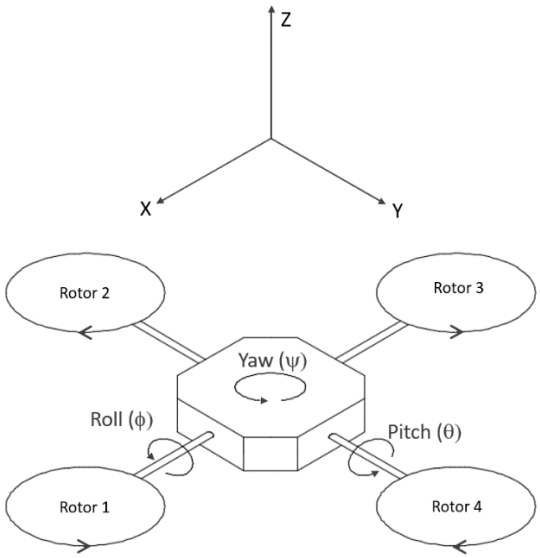



*Figure 1. Quadrotor coordinate system and sign convention.*

Quadrotor dynamics include coupled gyroscopic effects from rotation of both the quadrotor body and the rotors. For this project, simplified dynamics equations were used (Eq. 1, below). The main simplifying assumptions are: (1) rotor speeds change slowly enough that rotor acceleration is negligible, and (2) aerodynamic drag and other secondary effects are negligible. These simplified equations have been used by others in successful experiments [1] [2]. General translation in the x, y, and z directions was considered outside the scope of the project; however, transitional motion in the z direction was included since adjusting rotor speeds directly impacts the elevation. A representative state space model of the system dynamics includes eight states: $z, \phi, \theta, \psi, \dot{z}, \dot{\phi}, \dot{\theta}, and\ \dot{\psi}$.

$$
\begin{aligned}
\ddot{z} &= -g\, cos\phi\, cos\theta + \frac{1}{m} F_z \\
\ddot{\phi} &= \dot{\theta}\dot{\psi}\frac{J_y - J_z}{J_x} - \frac{J_r}{J_x}\,\dot{\theta}\Omega_r + \frac{1}{J_x}\, T_\phi \\
\ddot{\theta} &= \dot{\phi}\dot{\psi}\frac{J_z - J_x}{J_y} + \frac{J_r}{J_y}\dot{\phi}\Omega_r + \frac{1}{J_y}\, T_\theta \\
\ddot{\psi} &= \dot{\phi}\dot{\theta}\frac{J_x - J_y}{J_z} + \frac{1}{J_z}\, T_\psi
\end{aligned}
\qquad (1)
$$

Where,

$z$ = elevation, m
$\dot{z}$ = velocity in z direction, m/s
$\ddot{z}$ = acceleration in z direction, m/s$^2$
$\phi$ = roll angle, radians
$\dot{\phi}$ = roll rate, rad/s
$\ddot{\phi}$ = roll angular acceleration, rad/s$^2$
$\theta$ = pitch angle, radians
$\dot{\theta}$ = pitch rate, rad/s
$\ddot{\theta}$ = pitch angular acceleration, rad/s$^2$
$\psi$ = yaw angle, radians
$\dot{\psi}$ = yaw rate, rad/s
$\ddot{\psi}$ = yaw angular acceleration, rad/s$^2$
$J_x$ = quadrotor x axis moment of inertia, kgm$^2$
$J_y$ = quadrotor y axis moment of inertia, kgm$^2$
$J_z$ = quadrotor z axis moment of inertia, kgm$^2$
$J_r$ = single rotor moment of inertia, kgm$^2$
$\Omega_r$ = sum of rotor speeds $(-\Omega_1 + \Omega_2 - \Omega_3 + \Omega_4)$, rad/s
$g$ = gravitation constant, m/s$^2$
$m$ = mass, kg
$F_z$ = force in z direction, N
$T_\phi$ = torque about x axis, Nm

$T_\theta$ = torque about y axis, Nm
$T_\psi$ = torque about z axis, Nm

In the first line in Eq. 1 the *z* direction was fixed to the quadrotor body frame rather than the earth frame. Constants were used for the inertia terms to simplify formulas in the computer code:

$a_1 = (J_y - J_z) / J_x$, $a_2 = -J_r / J_x$, $a_3 = (J_z - J_x) / J_y$, $a_4 = J_r / J_y$, $a_5 = (J_x - J_y) / J_z$

Control inputs for this project are chosen to be $F_z$, $T_\phi$, $T_\theta$, and $T_\psi$, rather than voltages to the motors, in order to limit the scope of the project; however, rotor speeds are required for the dynamics equations and are also selectively included in the discussion of performance results.

Rotor speeds are related to $F_z$ and torques as follows.

$$\begin{aligned} F_z &= b\,(\Omega_1^2 + \Omega_2^2 + \Omega_3^2 + \Omega_4^2) \\ T_\phi &= L\,b\,(\Omega_4^2 - \Omega_2^2) \\ T_\theta &= L\,b\,(\Omega_3^2 - \Omega_1^2) \\ T_\psi &= d\,(\Omega_1^2 + \Omega_3^2 - \Omega_2^2 - \Omega_4^2) \end{aligned} \qquad (2)$$

Where,

$\Omega_1, \Omega_2, \Omega_3, \Omega_4$ = rotor speeds, rad/s
*b* = thrust factor, $Ns^2$
*L* = distance from quadrotor center to rotor, m
*d* = yaw rate drag factor, $Nms^2$

Solving Eq. 2 for rotor speeds yields

$$\begin{bmatrix} \Omega_1^2 \\ \Omega_2^2 \\ \Omega_3^2 \\ \Omega_4^2 \end{bmatrix} = \begin{bmatrix} 0.25 & 0 & -0.5 & 0.25 \\ 0.25 & -0.5 & 0 & -0.25 \\ 0.25 & 0 & 0.5 & 0.25 \\ 0.25 & 0.5 & 0 & -0.25 \end{bmatrix} \begin{bmatrix} F_z \\ T_\phi \\ T_\theta \\ T_\psi \end{bmatrix} \qquad (3)$$

Since the state space system includes eight states, auxiliary state stability equations were employed.

$$
\begin{aligned}
x_1 &= \Lambda z + \dot{z} \\
x_2 &= \Lambda \phi + \dot{\phi} \\
x_3 &= \Lambda \theta + \dot{\theta} \\
x_4 &= \Lambda \psi + \dot{\psi}
\end{aligned}
\qquad (4)
$$

Where,

$\Lambda$ = a (gain) constant, 1/s

Regulation of these auxiliary states results in stability in each of the original states. Equating the derivatives of the auxiliary states to the dynamics equations (Eq. 1) results in the following:

$$
\begin{aligned}
\dot{x}_1 &= \Lambda \dot{z} + \ddot{z} = \Lambda \dot{z} - g\, cos\phi\, cos\theta + \frac{1}{m} F_z \\
\dot{x}_2 &= \Lambda \dot{\phi} + \ddot{\phi} = \Lambda \dot{\phi} + \dot{\theta}\dot{\psi} a_1 + \dot{\theta}\Omega_r a_2 + \frac{1}{J_x} T_\phi \\
\dot{x}_3 &= \Lambda \dot{\theta} + \ddot{\theta} = \Lambda \dot{\theta} + \dot{\phi}\dot{\psi} a_3 + \dot{\phi}\Omega_r a_4 + \frac{1}{J_y} T_\theta \\
\dot{x}_4 &= \Lambda \dot{\psi} + \ddot{\psi} = \Lambda \dot{\psi} + \dot{\phi}\dot{\theta} a_5 + \frac{1}{J_z} T_\psi
\end{aligned}
\qquad (5)
$$

A modification was made the third equation in Eq. 5 to allow tracking of a trajectory. The auxiliary state was replaced with an auxiliary state error relating actual to desired terms.

$$
\begin{aligned}
z_3 &= x_3 - x_{3\,des} \\
\dot{z}_3 &= (\Lambda \dot{\theta} + \ddot{\theta}) - (\Lambda \dot{\theta_{des}} + \ddot{\theta_{des}}) \\
\dot{z}_3 &= \Lambda\left(\dot{\theta} - \dot{\theta_{des}}\right) + (\ddot{\theta} - \ddot{\theta_{des}}) \\
\dot{\widetilde{x_3}} &= \dot{z}_3 = \Lambda\left(\dot{\theta} - \dot{\theta_{des}}\right) + \dot{\phi}\dot{\psi} a_3 + \dot{\phi}\Omega_r a_4 - \ddot{\theta_{des}} + \frac{1}{J_y} T_\theta
\end{aligned}
\qquad (6)
$$

Where,

$\theta_{des}$ = desired pitch angle, radians

$\dot{\theta}_{des}$ = desired pitch rate, rad/s

$\ddot{\theta}_{des}$ = desired pitch angular acceleration, rad/s$^2$

The resulting state space system is

$$\dot{x} = f(x) + Bu$$

$$\begin{bmatrix} \dot{x_1} \\ \dot{x_2} \\ \dot{\widetilde{x_3}} \\ \dot{x_4} \end{bmatrix} = \begin{bmatrix} \Lambda\dot{z} - g\,cos\phi\,cos\theta \\ \Lambda\dot{\phi} + \dot{\theta}\dot{\psi}a_1 + \dot{\theta}\Omega_r a_2 \\ \Lambda\left(\dot{\theta} - \dot{\theta_{des}}\right) + \dot{\phi}\dot{\psi}a_3 + \dot{\phi}\Omega_r a_4 - \ddot{\theta_{des}} \\ \Lambda\dot{\psi} + \dot{\phi}\dot{\theta}a_5 \end{bmatrix} + \begin{bmatrix} 1/m & 0 & 0 & 0 \\ 0 & 1/J_x & 0 & 0 \\ 0 & 0 & 1/J_y & 0 \\ 0 & 0 & 0 & 1/J_z \end{bmatrix} \begin{bmatrix} F_z \\ T_\phi \\ T_\theta \\ T_\psi \end{bmatrix} \quad (7)$$

# 3 CONTROL AND ADAPTATION LAWS

A suitable Lyapunov function candidate for the state space system in Eq. 7 is

$$V = \frac{1}{2}x^T B^{-1}x + \frac{1}{2\beta}\,\widetilde{w}^T\widetilde{w} \quad (8)$$

Where,

β = adaptation gain
$\widetilde{w}$ = weights error
$\widetilde{w} = w - \widehat{w}$
$\dot{\widetilde{w}} = -\dot{\widehat{w}}$
w = ideal weights
$\widehat{w}$ = weights estimate

Taking the derivative and applying appropriate substitutions

$$\begin{aligned} \dot{V} &= \frac{1}{2}(\dot{x}^T B^{-1}x + x^T B^{-1}\dot{x}) + \frac{1}{2\beta}\,(\dot{\widetilde{w}}^T\widetilde{w} + \widetilde{w}^T\dot{\widetilde{w}}) \\ &= x^T B^{-1}\dot{x} + \frac{1}{\beta}\,\widetilde{w}^T\dot{\widetilde{w}} \\ &= x^T B^{-1}\,(\,f(x) + Bu\,) - \frac{1}{\beta}\,\widetilde{w}^T\dot{\widehat{w}} \\ &= x^T(\,B^{-1}\,f(x) + u\,) - \frac{1}{\beta}\,\widetilde{w}^T\dot{\widehat{w}} \end{aligned} \quad (9)$$

Approximation of the nonlinear term $B^{-1}f(x)$ was accomplished using a neural network algorithm, which is discussed in Section 5. The approximation can be represented symbolically as

$$B^{-1}f(x) = \Gamma(x)\, w + d(x)$$
$$= \textit{neural network approximation} + \textit{approximation error} \quad (10)$$

Where,

$\Gamma(x)$ = basis functions

$d(x)$ = approximation error

Combining Eq. 10 and the last line in Eq. 9 results in

$$\dot{V} = x^T(\ \Gamma(x)w + d(x) + u\ ) - \frac{1}{\beta}\,\tilde{w}^T\dot{\hat{w}} \quad (11)$$

A suitable control law that eliminates the nonlinear term and adds linear feedback is

$$u = -\ \Gamma(x)\hat{w} - Kx \quad (12)$$

Where,

K = matrix (diagonal) of feedback gains

*x* = auxiliary states ($x_1$, $x_2$, $x_3$, $x_4$)

The feedback gains K, combined with the gain $\Lambda$ in the state equations (Eq. 5), will result in a state feedback with full control of the poles of the associated characteristic equations.

Adding the control signal *u* to Eq. 11 results in

$$\begin{aligned}
\dot{V} &= x^T\ (\ \Gamma(x)\, w + d(x) - \Gamma(x)\,\hat{w} - K\,x\ )\ -\ \frac{1}{\beta}\,\tilde{w}^T\dot{\hat{w}} \\
&= x^T\ (\ \Gamma(x)\tilde{w} + d(x) - K\,x\ )\ -\ \frac{1}{\beta}\,\tilde{w}^T\dot{\hat{w}} \\
&= x^T\Gamma(x)\tilde{w} + x^T d(x) - x^T Kx\ -\ \frac{1}{\beta}\,\tilde{w}^T\dot{\hat{w}} \\
&= x^T d(x) - x^T Kx +\ \tilde{w}^T\Gamma^T(x)x - \frac{1}{\beta}\,\tilde{w}^T\dot{\hat{w}} \\
&= x^T d(x) - x^T Kx +\ \tilde{w}^T\ (\ \Gamma^T(x)x - \frac{1}{\beta}\,\dot{\hat{w}}\ )
\end{aligned} \quad (13)$$

The E-Modification adaptation method was selected with the corresponding robust weight updates as

$$\dot{\widehat{w}} = \beta \left( \Gamma^T(x)x - \nu |x| \widehat{w} \right) \quad (14)$$

Where,

ν = robust weights update constant

Inserting Eq. 14 into the last line in Eq. 13 results in

$$\dot{V} = x^T d(x) - x^T K x + \widetilde{w}^T \left( \Gamma^T(x)x - \frac{1}{\beta} \beta \left( \Gamma^T(x)x - \nu |x| \widehat{w} \right) \right)$$

$$\dot{V} = x^T d(x) - x^T K x + \widetilde{w}^T \nu |x| \widehat{w} \quad (15)$$

$$\dot{V} = x^T d(x) - x^T K x + \widetilde{w}^T \nu |x| w - \widetilde{w}^T \nu |x| \widetilde{w}$$

# 4 PROOF OF STABILITY

In a worst case scenario, and taking norms, the Lyapunov derivative result from Eq. 15 becomes

$$\dot{V} \leq \|x\| d_{max} - K_{min} \|x\|^2 + \nu \|x\| \|\widetilde{w}\| \|w\| - \nu \|x\| \|\widetilde{w}\|^2 \quad (16)$$

In Eq. 16, $d_{max}$ represents the maximum value of the estimation errors for each of the four function approximations, and $K_{min}$ is the minimum eigenvalue in the diagonal gains matrix *K*, or equivalently the minimum gain value.

For convenience we can make variable exchanges $X = \|x\|$, $D = D_{max}$, $K = K_{min}$, $\widetilde{W} = \|\widetilde{w}\|$, and $W = \|w\|$.

$$\dot{V} \leq X \left( D - KX + \nu \widetilde{W} W - \nu \widetilde{W}^2 \right) \quad (17)$$

Completing the square yields

$$\dot{V} \leq X \left( D - KX - \nu \left( \widetilde{W} - \frac{W}{2} \right)^2 + \frac{\nu W^2}{4} \right) \quad (18)$$

From Eq. 18, $\dot{V} < 0$ (which is what we want for stability) when $X$ and $\widetilde{W}$ exceed certain minimum values. This defines a Lyapunov surface, which is a uniform ultimate bound (UUB) for $\|x\|$ and $\|\widetilde{w}\|$ as shown in Fig. 2.

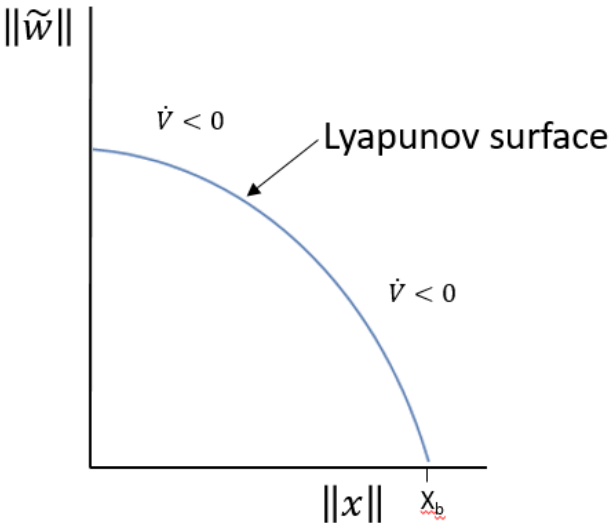



*Figure 2. Stability and UUB for $x$ and $\tilde{w}$.*

Minimum values of $\|x\|$ and $\|\tilde{w}\|$, denoted $\delta_x$ and $\delta_w$, for $\dot{V} < 0$ are calculated in Eq. 19 and 20, respectively

$$K\delta_x = D + \frac{\nu\ W^2}{4}$$

$$\delta_x = \frac{D}{K} + \frac{\nu\ W^2}{4\,K} \qquad (19)$$

$$\nu\left(\delta_w - \frac{W}{2}\right)^2 = D + \frac{\nu\ W^2}{4}$$

$$\left(\delta_w - \frac{W}{2}\right)^2 = \frac{D}{\nu} + \frac{W^2}{4} \qquad (20)$$

$$\delta_w = \frac{W}{2} + \sqrt{\frac{D}{\nu} + \frac{W^2}{4}}$$

To find the general magnitude of $X_b$ in Fig. 2, we can substitute the equations for $\delta_x$ and $\delta_w$ into Eq. 8 and solve for $X=X_b$ when $\tilde{W} = 0$

$$V(\delta_x, \delta_w) = V(X = X_b, \widetilde{W} = 0)$$

$$\frac{1}{2}\delta_x B^{-1}\delta_x + \frac{1}{2\beta}\delta_w\delta_w = \frac{1}{2}X_b B^{-1}X_b + 0 \qquad (21)$$

$$X_b \propto \sqrt{{\delta_x}^2 + \frac{1}{\beta}{\delta_w}^2 B}$$

Thus, on comparison of the result in Eq. 21 with that in Eq. 19 and 20, the following relationship is found

$$X_b \propto \frac{1}{K} \quad and \quad X_b \propto \nu \qquad (22)$$

# 5 The Cerebellar Model Articulation Controller (CMAC) Neural Network

CMAC has been described by others a simple model of the cortex of the cerebellum. It is both a neural network and a form of lookup table. Since the virtual memory requirements for this algorithm would be very large for a large (greater than three, say) number of inputs, the CMAC takes advantage of hash coding to map the virtual memory to a smaller space of physical memory, thus functioning as a lookup table.

A CMAC algorithm is used to approximate the nonlinear functions that result from derivation of the control law and given by the RHS of Eq. 23

$$\begin{bmatrix} f_1(x) \\ f_2(x) \\ f_3(x) \\ f_4(x) \end{bmatrix} = B^{-1}f(x) = \begin{bmatrix} \mathrm{m}\,(\Lambda\dot{z} - g\,cos\phi\,cos\theta) \\ J_x\,(\Lambda\dot{\phi} + \dot{\theta}\dot{\psi}a_1 + \dot{\theta}\Omega_r a_2) \\ J_y\left(\Lambda\left(\dot{\theta} - \dot{\theta_{des}}\right) + \dot{\phi}\dot{\psi}a_3 + \dot{\phi}\Omega_r a_4 - \ddot{\theta_{des}}\right) \\ J_z\,(\Lambda\dot{\psi} + \dot{\phi}\dot{\theta}a_5) \end{bmatrix} \qquad (23)$$

## 5.1 Offline (static) Training

For Project 1, Multi-layer Perceptron (MLP) was initially used for static offline training of the nonlinear terms ($f_1$, $f_2$, $f_3$, and $f_4$) given by the RHS of Eq. 23. The input variables are $\dot{z}, \phi, \text{and } \theta$ for $f_1$, $\dot{\phi}, \dot{\theta}, \dot{\psi}, and\ \Omega_r$ for $f_2$, $\dot{\theta}, \dot{\theta_{des}}, \dot{\phi}, \dot{\psi}, \Omega_r, and\ \ddot{\theta_{des}}$ for $f_3$, and $\dot{\psi}, \dot{\phi}, and\ \dot{\theta}$ for $f_4$.

Suitable ranges for the input variables were established for the purpose of stability control and following predefined trajectories in the control simulation work and are shown in Table 1.

*Table 1. Input value ranges for nonlinear approximation.*

| Variable | Units | Minimum Value | Maximum Value |
|---|---|---|---|
| $\dot{z}$ | m/s | -5 | 5 |
| $\phi, \theta$ | rad | $-\pi/2$ | $\pi/2$ |
| $\dot{\phi}, \dot{\theta}, \dot{\psi}$ | rad/s | -2.5 | 2.5 |
| $\Omega_r$ | rad/s | -50 | 50 |
| $\dot{\theta}_{des}$ | rad/s | -2.5 | 2.5 |
| $\ddot{\theta}_{des}$ | rad/s$^2$ | -2.5 | 2.5 |

The $f_1$ approximation gave large error using MLP and so this approximation was subsequently updated using CMAC with improved results. Subsequent to Project 1, the remaining three nonlinear functions were approximated using CMAC, with significantly improved error performance as shown below:

*Table 2. Comparison of MLP and CMAC static (offline) training of nonlinear terms.*

| Nonlinear Term | MLP | | CMAC | |
|---|---|---|---|---|
| | RMS Error | $d_{max}$ | RMS Error | $d_{max}$ |
| $f_1$ | 5 | 10 | 0.12 | 1.5 |
| $f_2$ | 0.5 | 1.7 | 0.007 | 0.1 |
| $f_3$ | 0.5 | 1.8 | 0.035 | 0.45 |
| $f_4$ | 0.2 | 0.5 | 0.0025 | 0.03 |

Subsequent to the initial CMAC update, a correction was made to the formula for $\Omega_r$. As a result, the final RMS error and $d_{max}$ values used are lower than those in Table 2 for the $f_2$ and $f_3$ terms. Plots showing the error convergence (with final CMAC results) are shown in Appendix I. The weights resulting from the offline training were used as a starting point for simulation runs in the current project.

The static training CMAC algorithm uses 20 layers, a $\beta$ value of 0.1, and 10 quantizations for $f_1$ and $f_4$ and 5 quantizations for $f_2$ and $f_3$. The smaller number of quantizations used for $f_2$ and $f_3$ relate to the larger number of inputs and using a trial and error approach to obtaining convergence results within a reasonable timeframe of algorithm runtime.

## 5.2 Online Training (Adaptation)

An advantage of CMAC over the Multi-layer Perceptron (MLP) during online training is that only the weights on activated cells (on all layers) are updated at each time step. For the MLP, all hidden and output weights need to be updated at each time step since the output calculation is dependent on all of these weights. In this respect the CMAC may offer an advantage of speed.

A majority of the simulation runs use the weights obtained from the offline training as a starting point. In the remaining cases, the CMAC algorithm is initiated with weights set at zero. When initial weights are

set to zero, the online training CMAC algorithm uses 50 layers and 10 quantizations for all function approximations. Input value ranges for online training are the same as for the offline training (Table 1).

# 6 CLOSED LOOP CONTROL SIMULATION

Although simulation modelling for Project 1 was done primarily in Simulink, the current project uses only MATLAB® code, with the ODE45 function called to solve the differential equations. The switch was made because the MATLAB® code offered low level programming flexibility.

An issue that arises when solving the differential equations (Eq. 1) in runtime is that the nonlinear function approximation component (Eq. 23) of the control inputs ($F_z$, $T_\phi$, $T_\theta$, and $T_\psi$) depends on the rotor speeds. However, in turn the rotor speeds depend on the control inputs (Eq. 3). This analytical loop situation is resolved by storing and using the previous time step value for sum of rotor speeds ($\Omega_r$). This approximation will introduce a disturbance to the system; however, due to the small time step used (0.01s) it assumed to be negligible.

Quadrotor parameters used in the simulation runs are the same as for Project 1, with an exception for the mass, which is altered in select cases to simulate a disturbance. Parameters are shown in Table 3.

*Table 3. Quadrotor physical parameters.*

| Parameter | Symbol | Value | Units |
|---|---|---|---|
| Quadrotor moment of inertia in x direction | Jx | 0.01 | Kg $m^2$ |
| Quadrotor moment of inertia in y direction | Jy | 0.01 | Kg $m^2$ |
| Quadrotor moment of inertia in z direction | Jz | 0.02 | Kg $m^2$ |
| Single rotor moment of inertia | Jr | 0.0001 | Kg $m^2$ |
| Thrust factor | b | 6.13E-05 | N $s^2$ |
| Distance from quadrotor center to rotor | L | 0.25 | m |
| Yaw rate drag factor | d | 1.00E-06 | N m $s^2$ |
| Quadrotor mass (initial) | m | 1 | kg |

A summary of the closed loop simulation parameters is shown in Table 3, with performance plots and discussion in the various sections below the table.

*Table 4. Quadrotor closed-loop control simulation parameters.*

| Parameter | Simulation Case | | | | | |
|---|---|---|---|---|---|---|
| | **1A** | **1B** | **2A** | **2B** | **3** | **4** |
| **Initial conditions (elev., angles)** | all at 0 | all at 0 | all at 0 | all at 0 | all at 0 | z 0m<br>$\phi$ 20°<br>$\theta$ -20°<br>$\psi$ 20° |
| **Initial NN weights** | 0 | Pre-trained | 0 | Pre-trained | Pre-trained | Pre-trained |
| *Feedback gains* | | | | | | |
| **K** | [5 2 2 2] | [5 2 2 2] | [5 2 2 2] | [5 2 2 2] | [5 2 2 2] | [5 2 2 2] |
| **$\Lambda$** | 2 | 2 | 2 | 2 | 2 | 2 |
| **Trajectory** | na | na | na | na | na | pitch axis oscillation (sinusoid)<br>amplitude 45°<br>period 4s<br>duration: continuous from 1s |
| **Disturbance** | mass reduced from 1.0 kg to 0.5 kg at 1s mark | mass reduced from 1.0 kg to 0.5 kg at 1s mark | wind torque about roll axis (sinusoid)<br>amplitude 0.5 Nm<br>center 0.2 Nm<br>period 1s<br>duration: from 1s to 20s then stop | wind torque about roll axis (sinusoid)<br>amplitude 0.5 Nm<br>center 0.2 Nm<br>period 1s<br>duration: from 1s to 20s then stop | mass reduced from 1.0 kg to 0.5 kg at 1s mark<br>wind torque about roll axis (sinusoid)<br>amplitude 0.5 Nm<br>center 0.2 Nm<br>period 1s<br>duration: from 1s to 20s then stop | mass reduced from 1.0 kg to 0.5 kg at 1s mark<br>wind torque about roll axis (sinusoid)<br>amplitude 0.5 Nm<br>center 0.2 Nm<br>period 1s<br>duration: from 1s to 20s then stop |
| **Adaptation Method** | E-Modification $\beta$=100, $\nu$=0.0011 | E-Modification $\beta$=100, $\nu$=0.0011 | E-Modification $\beta$=10, $\nu$=0.1 | E-Modification $\beta$=10, $\nu$=0.01 | E-Modification $\beta$=10, $\nu$=0.01 | E-Modification $\beta$=100, $\nu$=0.008 |

## 6.1 Simulation Case 1

The first closed loop simulation case was a simple test of the quadrotors ability to adapt to a change in mass. The quadrotor mass was initially set to 1.0kg. At the 1s mark, the quadrotor's mass was reduced to 0.5kg, thus simulating the dropping of a package for delivery, say. No trajectory was included and all initial conditions (elevation, angles) were set to 0.

Scenario A of this case assumes all initial weights are zero. For the results, Fig. 3 shows the change in elevation while Fig. 4 shows convergence of the elevation weights. Note that having zero initial weights is not a realistic condition for a quadrotor since we wish to start from a hovering state and zero weights implies rotor speeds will initially be zero. This is evident in Fig. 3 with the initial drop in elevation to about 0.75m below equilibrium prior to the 1s mark. At the 1s mark, elevation increases with the mass reduction to 0.5kg and returns to the equilibrium position at about the 10s mark. Weights converge fairly quickly with minor erratic behavior as shown in Fig. 4.

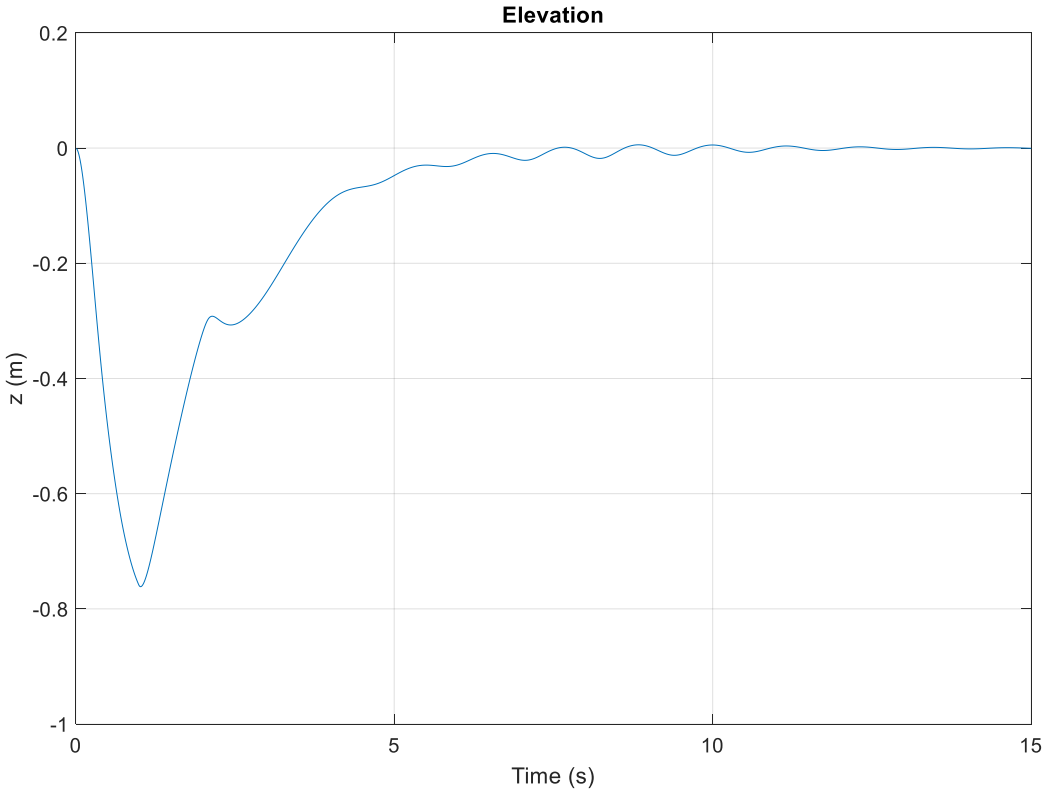



*Figure 3. Simulation case 1A; elevation; $\beta$=100, $\nu$=0.0011.*

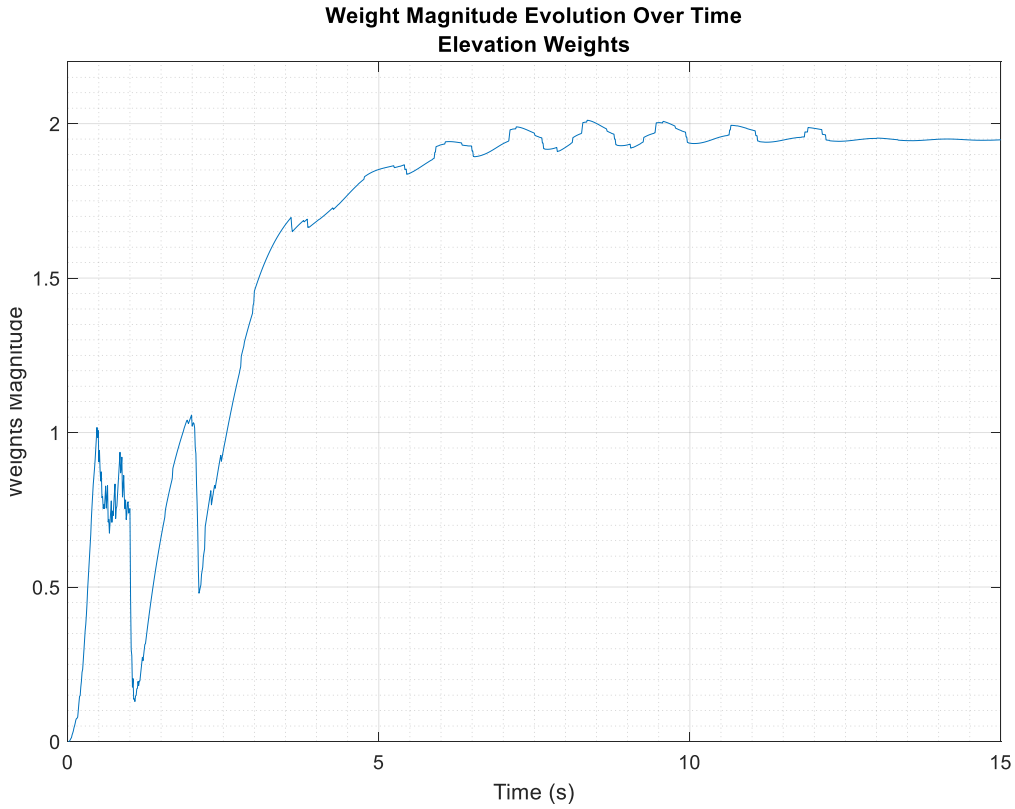



*Figure 4. Simulation case 1A; weights convergence; $\beta$=100, $\nu$=0.0011.*

Scenario B for Case 1 uses the pre-trained weights. All other parameters are the same. This time, elevation behaves as expected (Fig. 5): equilibrium maintained until the 1s mark and then an increase in elevation with the shedding of 0.5kg. A maximum elevation of 0.2m is reached at just under 1s after weight reduction, followed by some overshoot prior to settling at the 6s mark. Fig. 6 illustrates weights convergence. The interesting jogs in the curve are likely the result of the switching of cells and different basis function values in the CMAC algorithm.

On comparison of the results from Scenario A and B, it is evident that having non-zero initial weights, in other words, weights that are reflective of the quadrotor's actual physical parameters, results in improved performance.

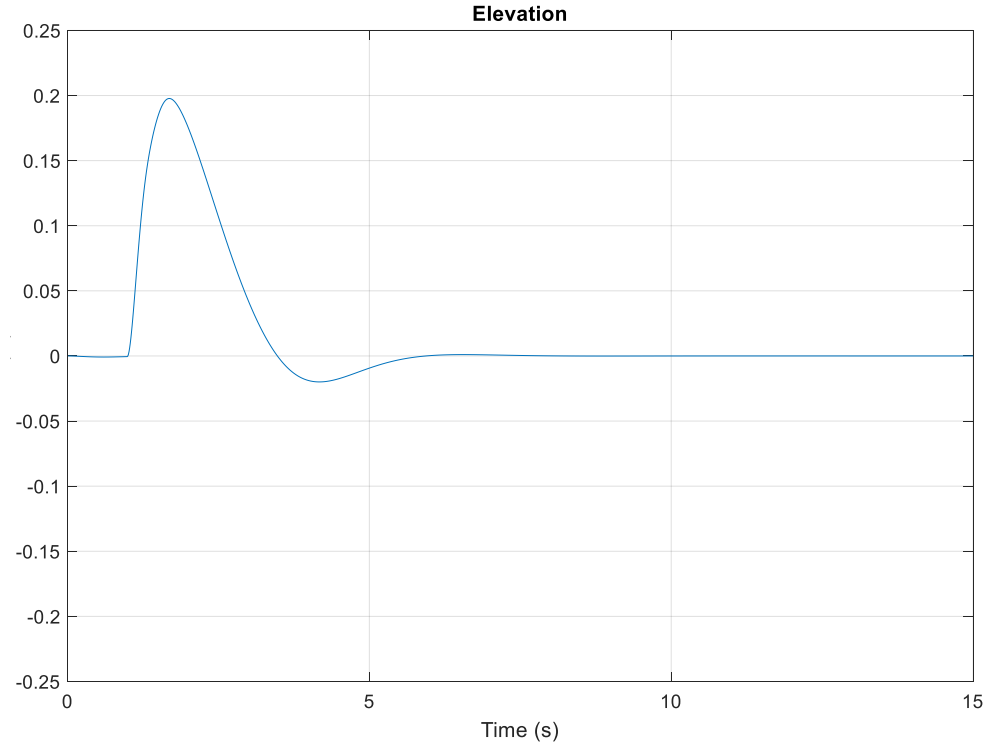



*Figure 5. Simulation case 1B; elevation; $\beta$=100, $\nu$=0.0011.*

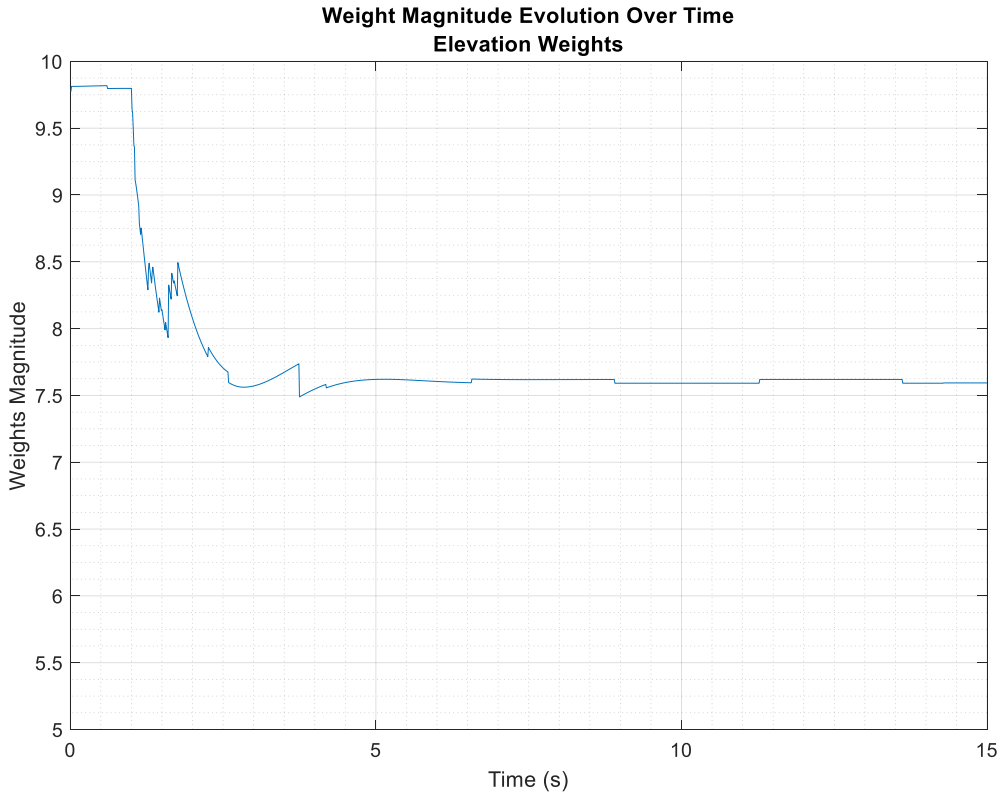


*Figure 6. Simulation case 1B; weights convergence; $\beta$=100, $\nu$=0.0011.*

## 6.2 SIMULATION CASE 2

The next closed loop simulation was performed to show the quadrotor's stability and robustness to a simulated wind disturbance. A sinusoidal torque with an average value of 0.2 Nm, amplitude of 0.5 Nm, and period of 1s is applied about the roll axis between t=1s and t=20s of simulation time, after which the oscillation stops and constant wind torque is maintained. Initial elevation, angles, and weights are zero. Also, no trajectory is applied and as a consequence the control is just trying to keep the quadrotor at steady equilibrium.

Senario A of Case 2 uses zero initial weights. For low values of $\nu$ (below 0.08), bursting of the weights occurs, likely a result of wind disturbance oscillations causing input variables to oscillate between cells close to the origin (Fig. 7) [2]. In this case the $\phi$ and $\theta$ weights continue to diverge after the disturbance stops at the 20s mark. As $\nu$ is increased, elevation weights converge to final values (Fig. 8, first plot); however, the elevation has also dropped below equilibrium and never returns (Fig. 8, second plot). Furthermore, weights convergence is misleading. The neural network component of control is small in comparison to the feedback component (Fig. 8, third and fourth plots). This situation was not resolvable for trials considering many different values of $\nu$.

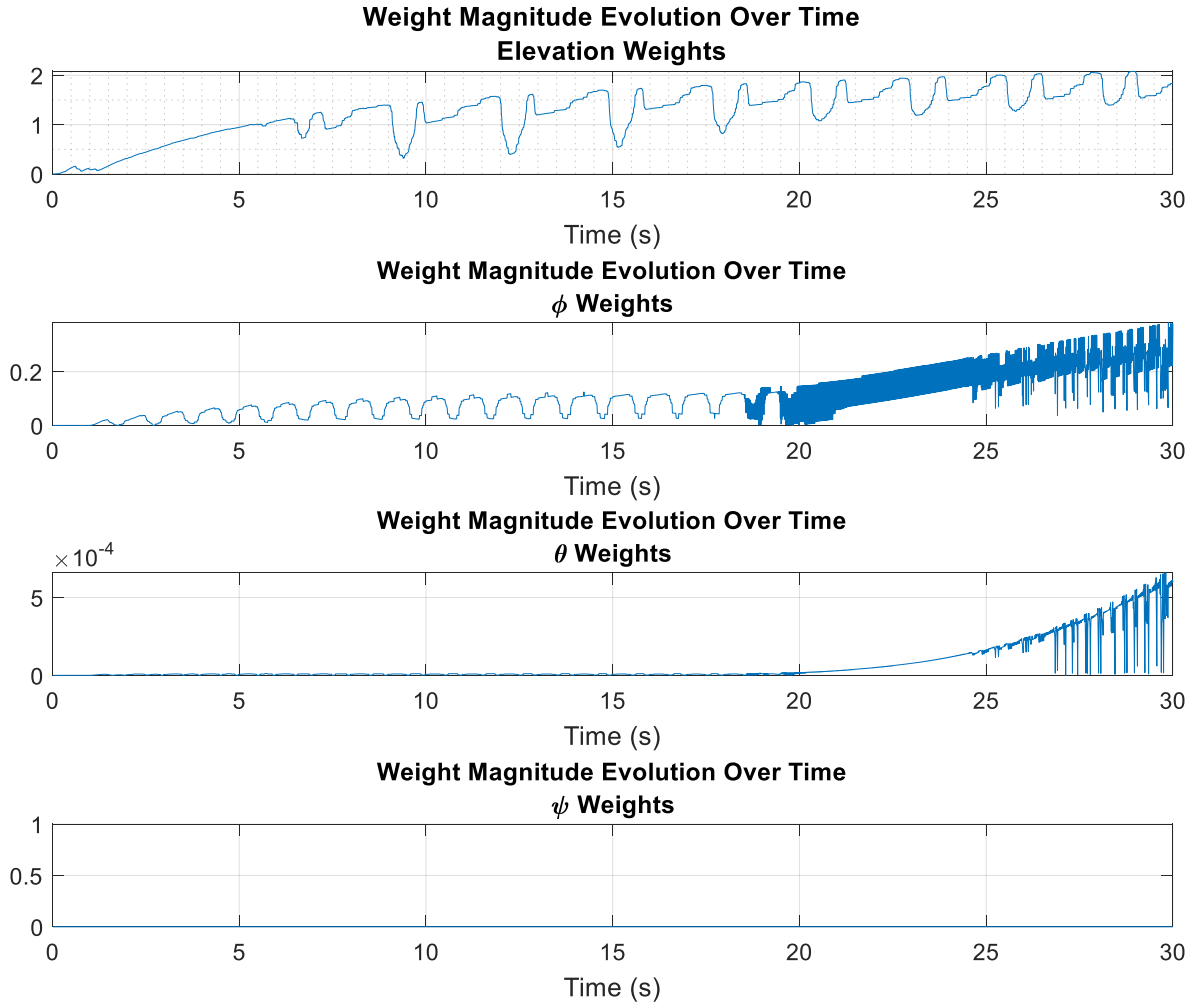



*Figure 7. Simulation case 2A; weights bursting with low $\nu$; $\beta$=10, $\nu$=0.01.*

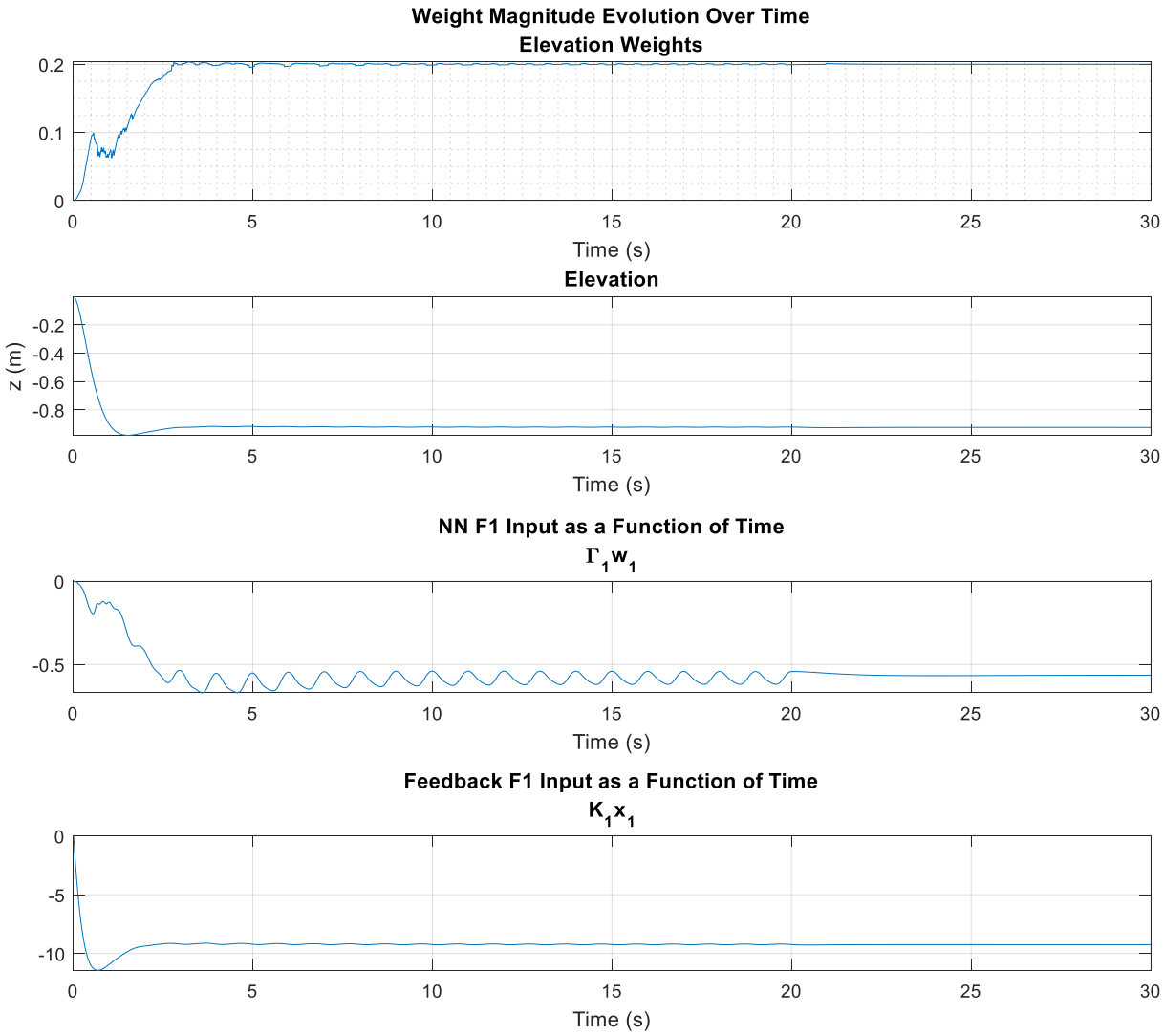



*Figure 8. Simulation case 2A; weights convergence but with steady state elevation error; $\beta$=10, $\nu$=0.1.*

Scenario B of Case 2 consists of the same simulation parameters as scenario A, but using the pre-trained initial weights. A roll oscillation with amplitude of about 2 degrees is observed as with Scenario A, but with the remaining angles and elevation steady at equilibrium (Fig. 9). Fig. 10 illustrates that rotors 2 and 4 are providing the torque to stabilize the roll axis. Since the roll angle oscillation is minor, the orthogonal pitch axis gyroscopic precession is minimal and so rotors 1 and 3 speeds are fairly steady.

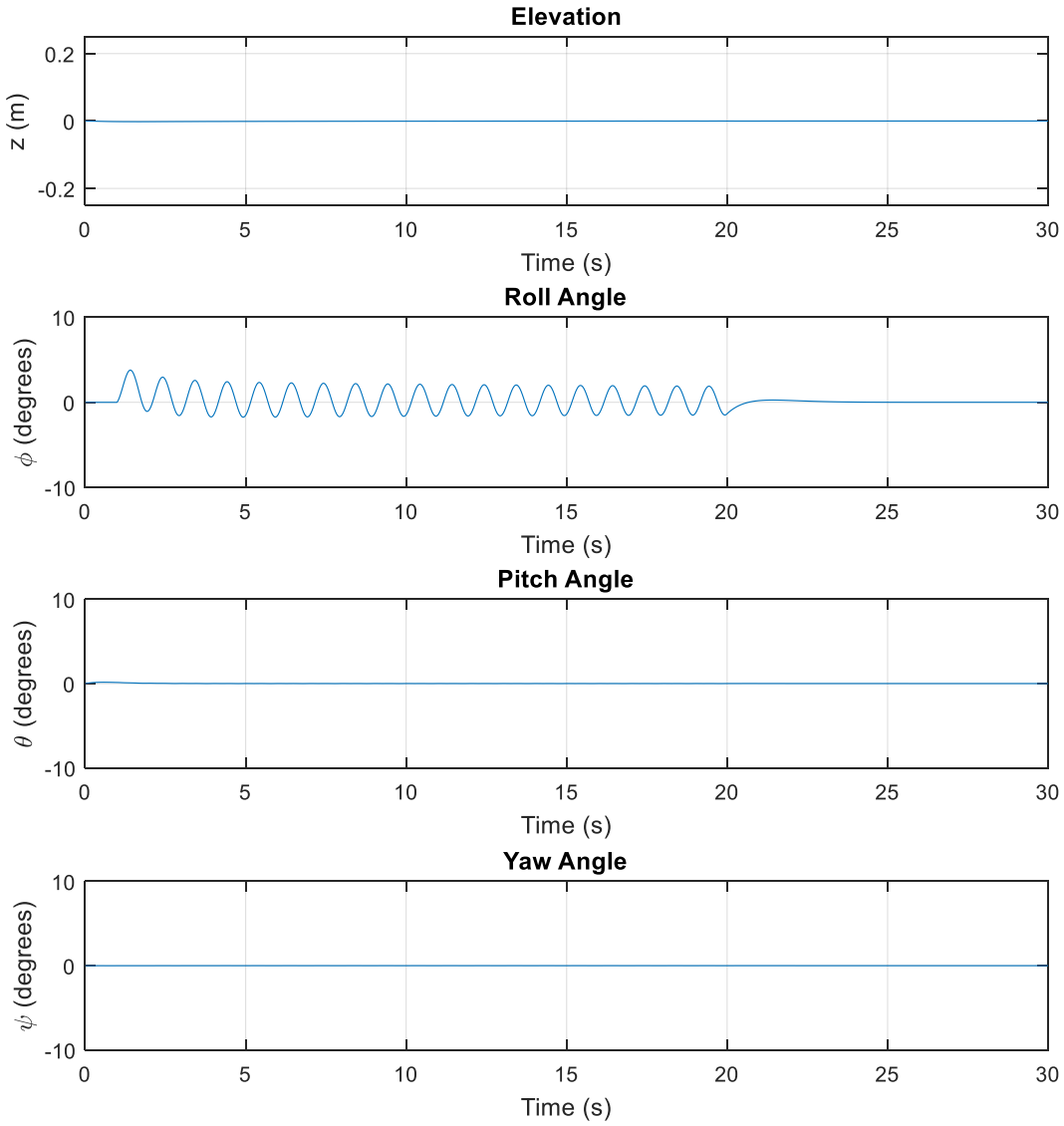



*Figure 9. Simulation case 2B; elevation and angles; $\beta$=10, $\nu$=0.01.*

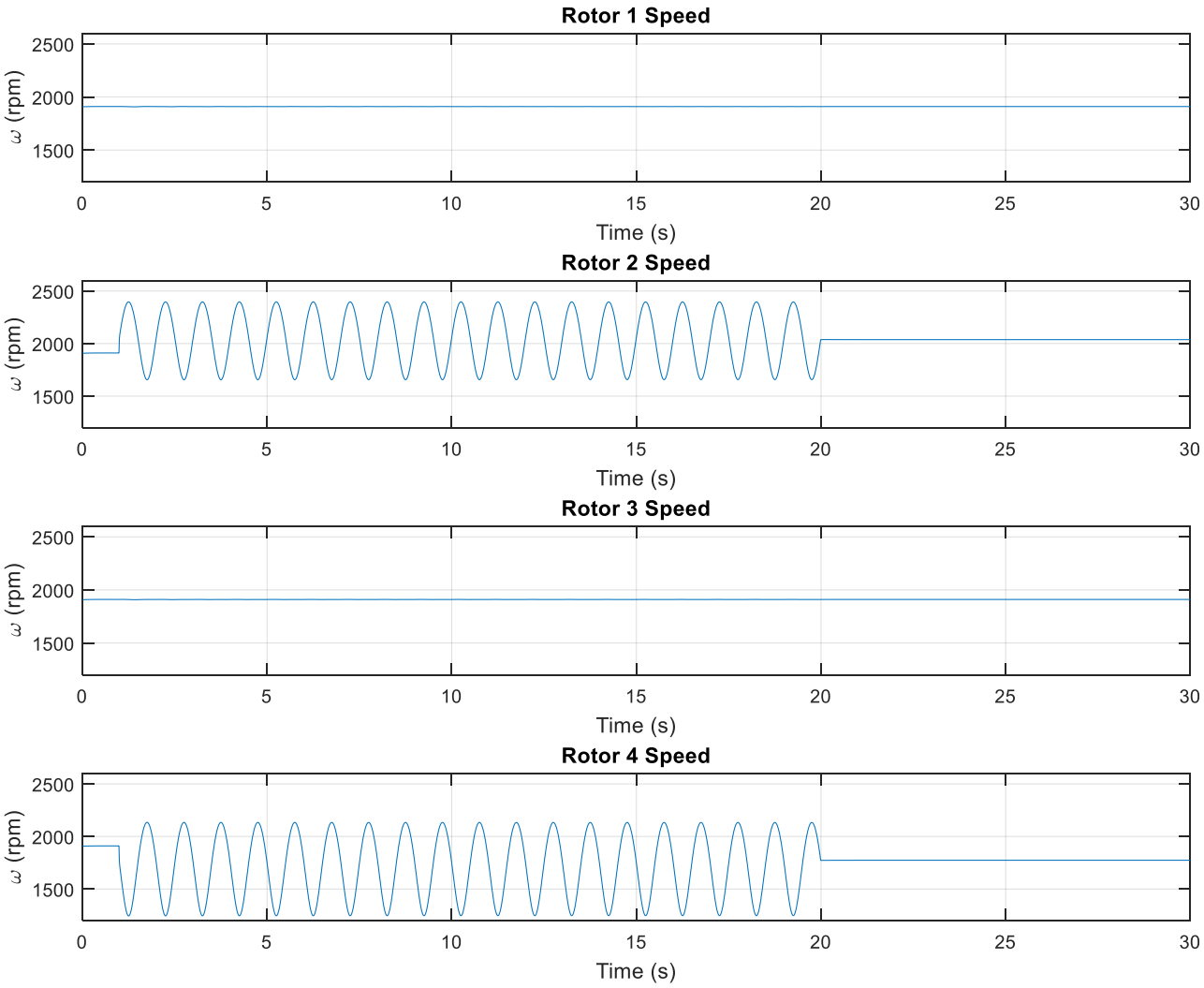



*Figure 10. Simulation case 2B; rotor speeds; $\beta$=10, $\nu$=0.01.*

Figure 11 illustrates weights convergence. Note the $\psi$ weights fluctuate significantly. This is likely from different CMAC cells being activated close to the equilibrium point, whereby the basis functions take on different values and weights are discontinuous; however, this behavior doesn't affect yaw stability. $\phi$ weights converge approximately 3 seconds after the disturbance stops, although it is evident that throughout the 1s to 20s period the weights gradually but continually grow in amplitude.

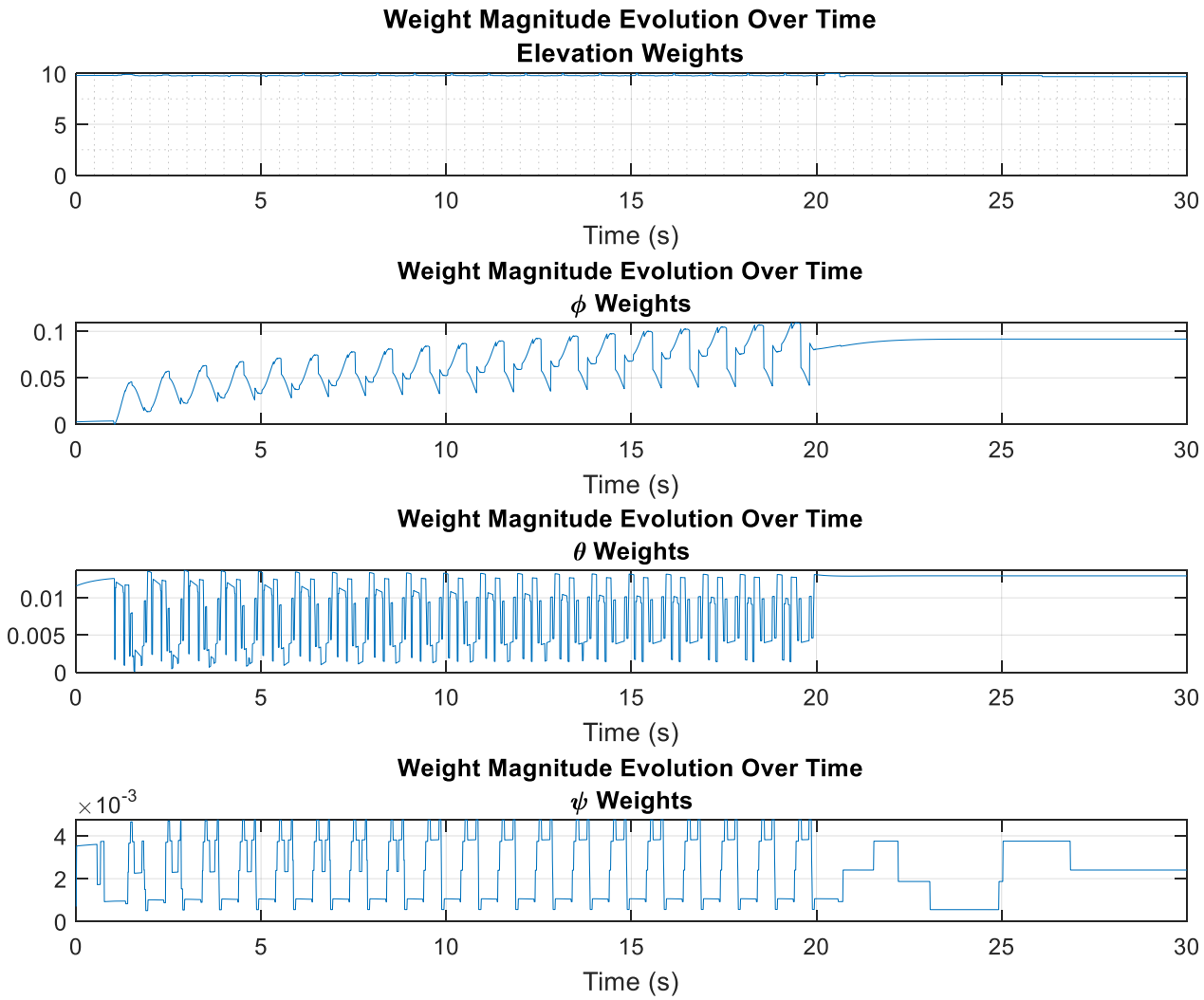



*Figure 11. Simulation case 2B; weights convergence; $\beta$=10, $\nu$=0.01.*

## 6.3 Simulation Case 3

The next case was performed with the combined disturbances of cases 1 and 2, namely both the reduction in mass and sinusoidal wind. Only the pre-trained weights are used, and therefore the results are somewhat predictable; plots showing stability and weights convergence are shown in Fig. 12 and 13, respectively.

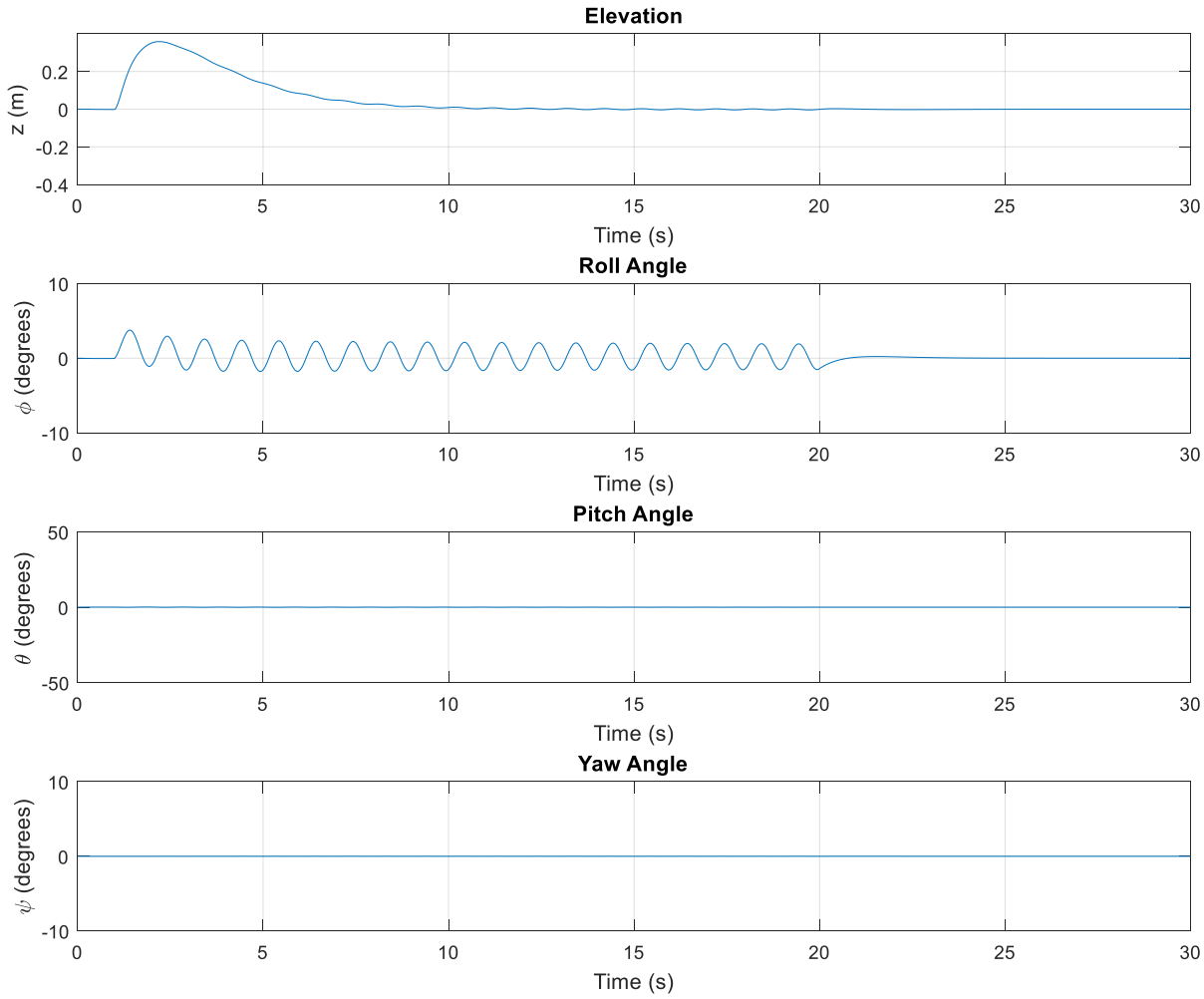



*Figure 12. Simulation case 3; elevation and angles; $\beta$=10, $\nu$=0.01.*

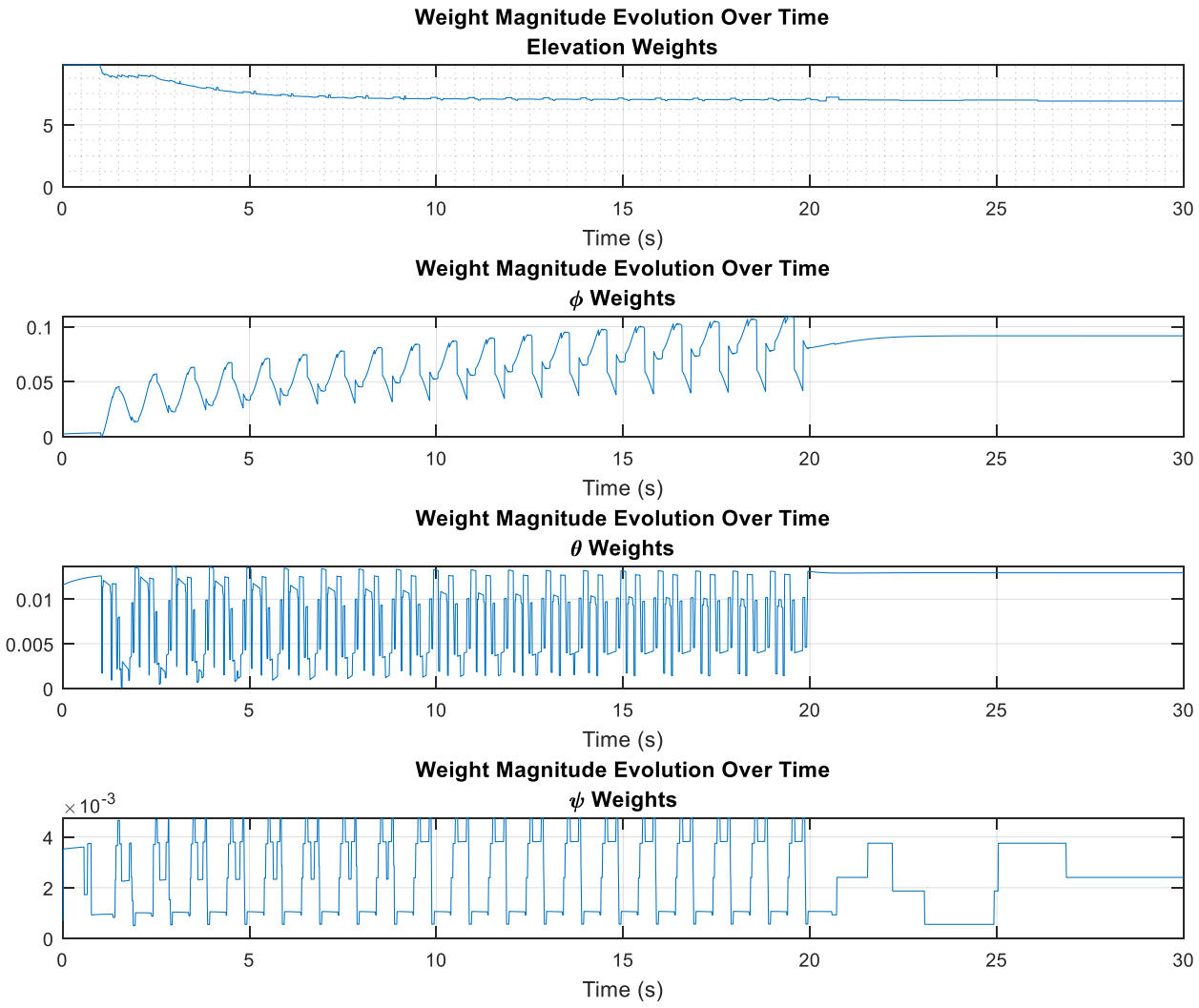



*Figure 13. Simulation case 3; weights convergence; $\beta$=10, $\nu$=0.01.*

## 6.4 Simulation Case 4

The last case considered combines disturbance cases 1 and 2 and adds a sinusoidal pitch trajectory of amplitude 45° and period 4s. The pre-trained weights are used and the trajectory is continuous throughout the duration of the test beginning at the 1s mark. Initial angles are non-zero to illustrate initial tracking convergence.

An adaptation gain ($\beta$) of 100 was selected, somewhat arbitrarily, for this case. A lower $\beta$ of 10 or 20 seemed to yield slower returns to equilibrium. A range of robust weights update constants ($\nu$) were tested. Low values of $\nu$, including the basic Leakage method with $\nu$ = 0, resulted in weights bursting.

Bursting stopped at approximately $\nu$ = 0.005. Increasing values of $\nu$, above approximately 0.05, resulted in a similar situation as Case 2 Scenario A: increasing steady state elevation error. Values of $\nu$ between 0.006 and 0.01 provided acceptable performance. A value of 0.008 was used for the simulation.

In Fig. 14, elevation increases at the 1s mark due to the reduction in mass and fluctuates during stabilization to equilibrium. This fluctuation relates to the oscillating rotor speeds that are countering mainly the wind disturbance and also the trajectory to a lesser extent. Roll angle fluctuates a minor amount from the wind disturbance as observed previously. Pitch angle adapts to the trajectory quickly (Fig 15).

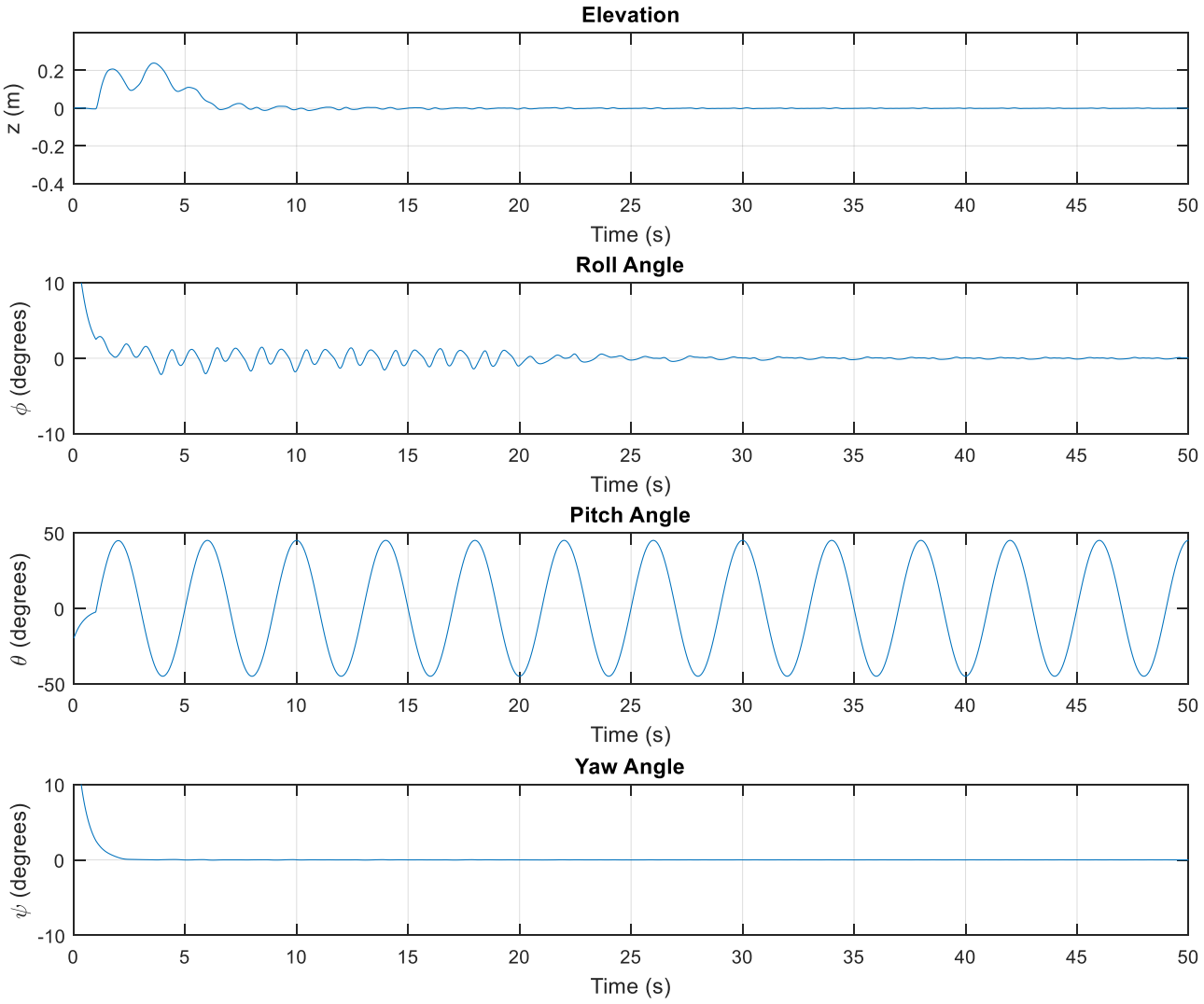



*Figure 14. Simulation case 4; elevation and angles; $\beta$=100, $\nu$=0.008.*

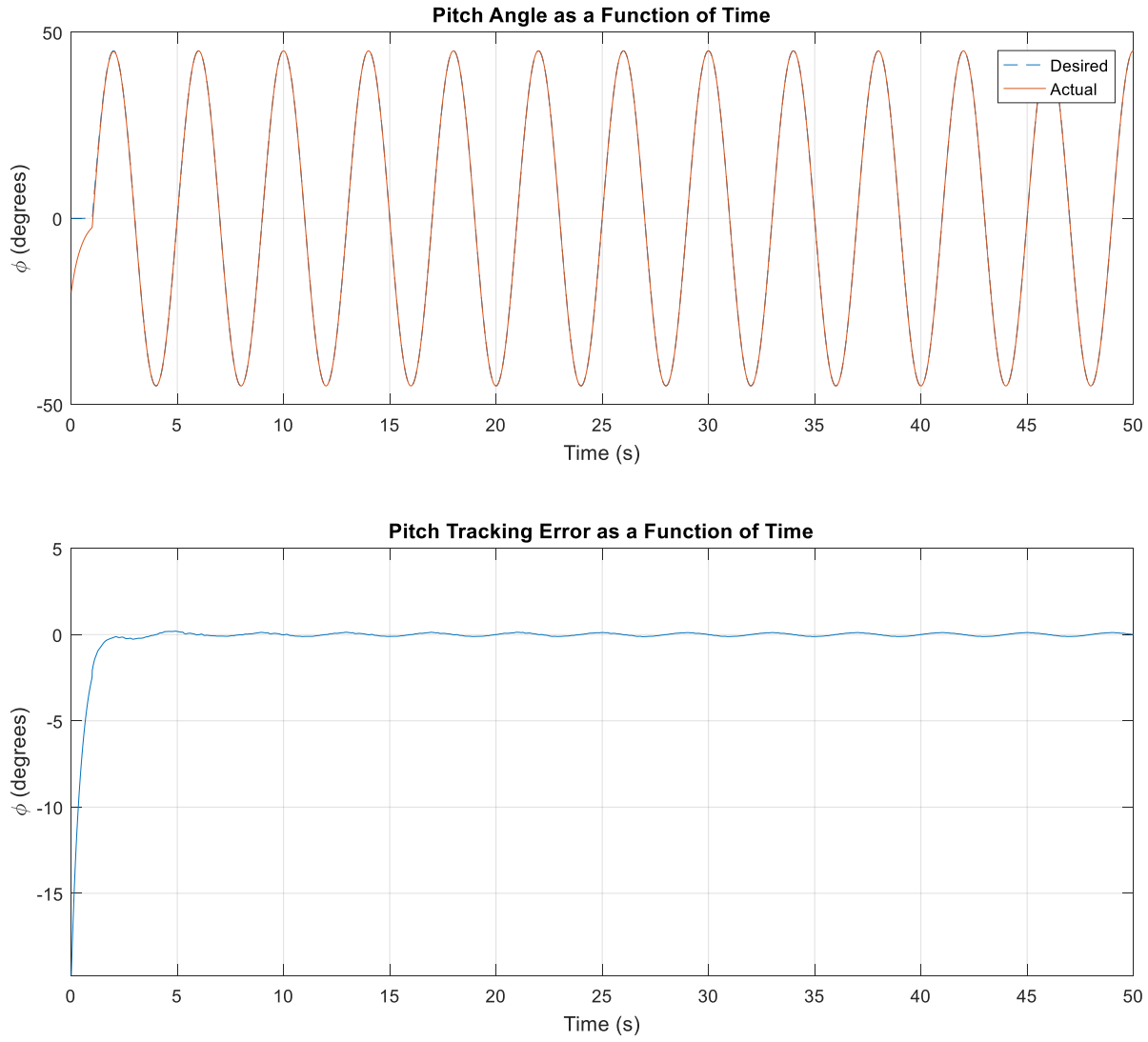



*Figure 15. Simulation case 4; pitch tracking; $\beta$=100, $\nu$=0.008.*

In Fig. 16, observe that the input torque about the X ($\phi$) axis is larger than the input torque about the Y ($\theta$) axis during the disturbance period. This shows that the wind disturbance torque is more demanding on the controls than the trajectory. That may seem counter intuitive; however, there are a couple reasons for this: (1) the disturbance torque is simply quite large, and (2) the period of the disturbance oscillation is four times shorter than the period of the trajectory oscillation. Higher torques associated with shorter periods are a direct consequence of the same idea whereby acceleration amplitudes are higher for oscillations with higher frequencies.

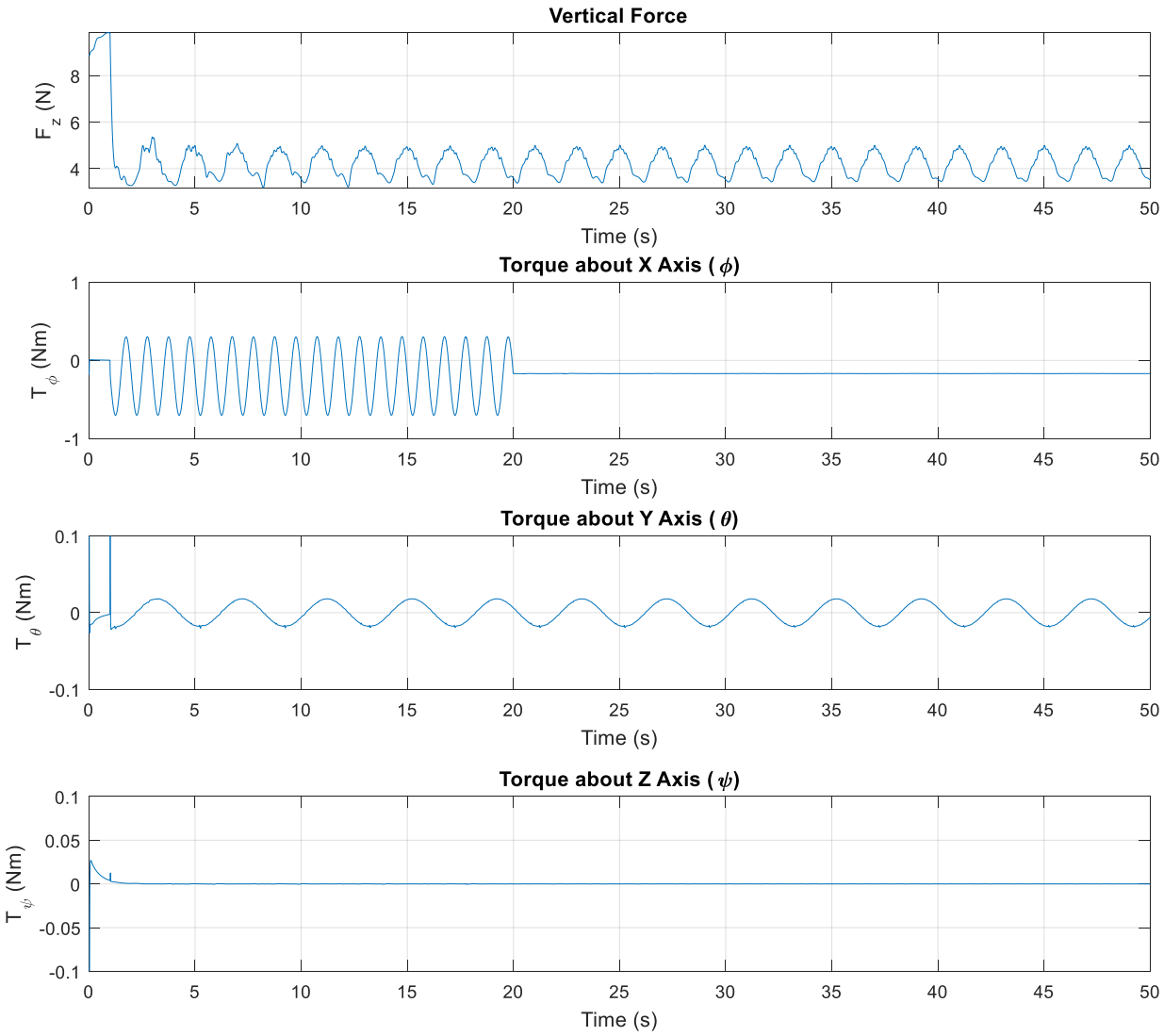



*Figure 16. Simulation case 4; vertical force and torques; $\beta$=100, $\nu$=0.008.*

Figure 17 depicts a measure of the weights magnitude over the simulation period. The character of the magnitudes changes immediately after the disturbance period at the 20s mark and then stabilizes with constant character thereafter.

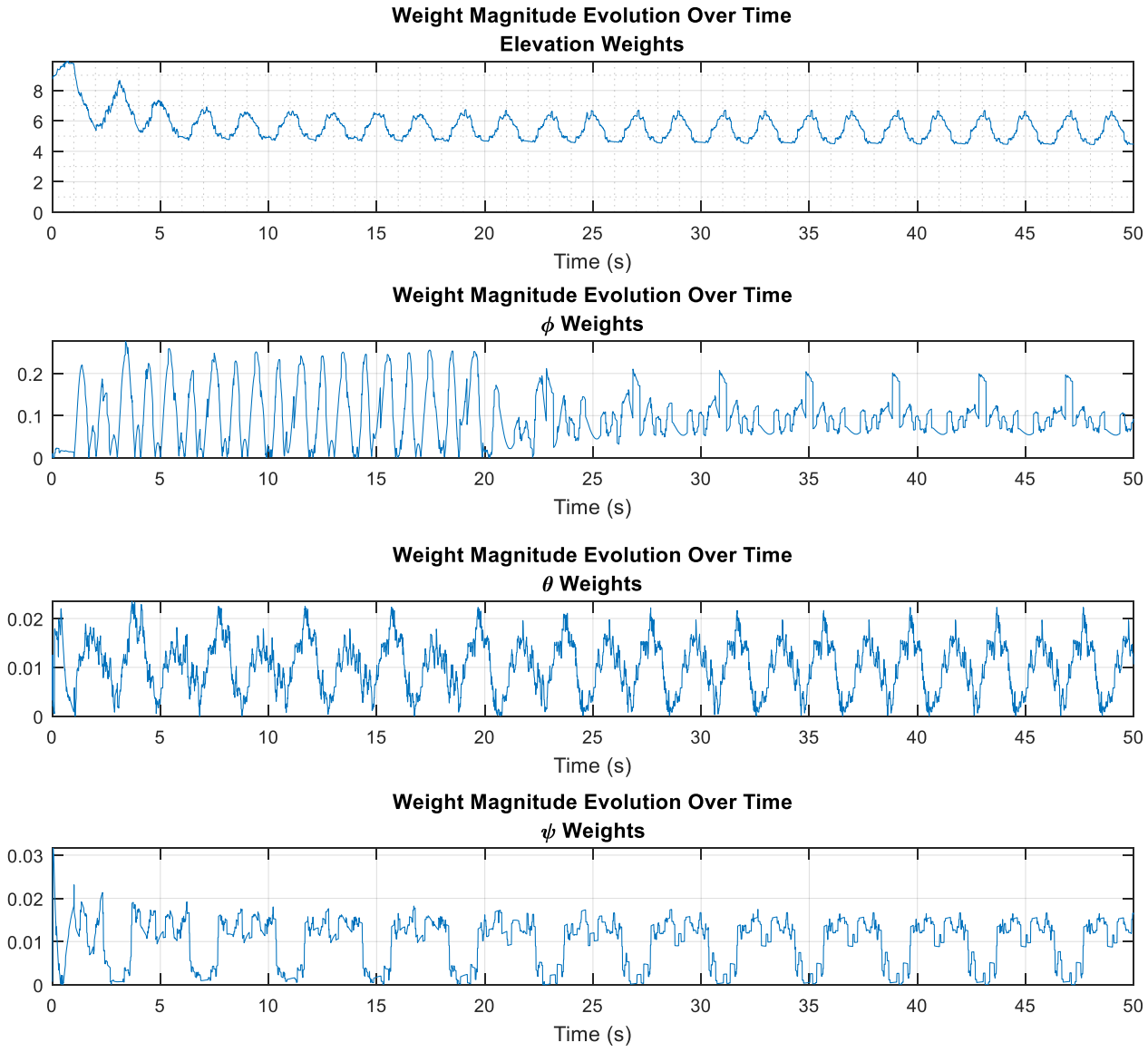



*Figure 17. Simulation case 4; weights convergence; $\beta$=100, $\nu$=0.008.*

# 7 CONCLUSIONS

Adaptive control of a quadrotor helicopter was achieved using the CMAC neural network algorithm. Both pre-trained neural network weights and weights initially at zero were used to illustrate adaptive performance in several simulated hover mode tests.

The first test was a disturbance involving a 50% reduction in mass to the quadrotor, confirming its ability to adapt with both pre-trained and zero initial weights, but with performance tradeoffs in elevation stability.

A simulated wind disturbance of limited duration was tested next. The simulated wind disturbance was a sinusoidal torque oscillation of 20s duration about the roll ($\phi$) axis. A constant oscillation disturbance is probably an extreme scenario. In reality the wind would likely behave less predictably but with "steady" periods mixed in with the oscillations. Disturbance rejection and stability was confirmed for the case of pre-trained initial weights, with weights converging to final values shortly following the disturbance. However, for the case of weights initially at zero, either the weights showed bursting behavior following the disturbance or the elevation had significant steady state error, depending on selection of $\nu$. There was no value of $\nu$ found that prevented either from occurring. This behavior seems to show that it is important to start with initial weights reflective of (at least approximate) quadrotor physical parameters. This requirement is also mentioned by others [3].

The third test was a combination of the first two test disturbances, using pre-trained initial weights, with positive results for disturbance rejection and weights convergence.

The final test, using pre-trained initial weights, included both types of disturbances as well as a sinusoidal pitch trajectory. This test confirmed, upon selection of adequate $\beta$ and $\nu$ parameters, the quadrotor's ability to respond quickly to and track a trajectory input and concurrently reject and adapt to disturbances.

Performance was found to be sensitive to the selected robust weights update parameter $\nu$. Small values of $\nu$ resulted in a bursting effect of the weights and subsequent instability in elevation and the quadrotor angles. Large values of $\nu$ caused steady state errors in elevation and other erratic behavior. Careful selection of $\nu$ was required to achieve acceptable performance.

# REFERENCES

[1] S. Bouabdallah and R. Siegwart, "Backstepping and Sliding-mode Techniques Applied to an Indoor Micro Quadrotor," in *International Conference on Robotics and Automation*, Barcelona, Spain, 2005.

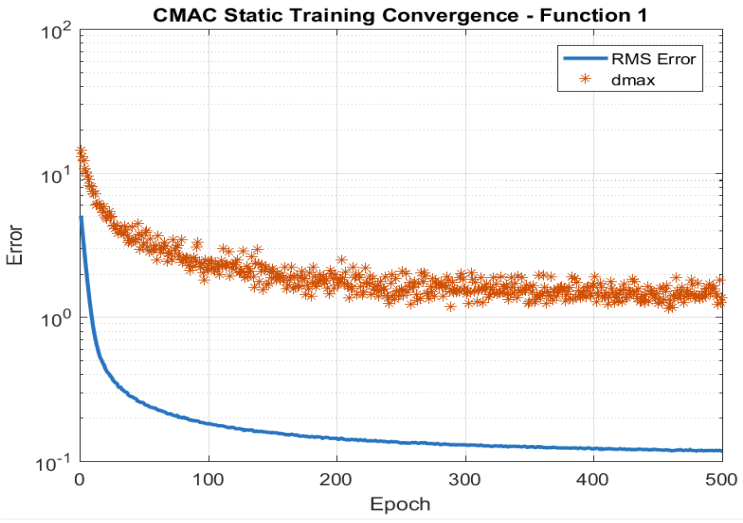
[2] C. Nicol, C. Macnab and A. Ramiriz-Serrano, "Robust Adapdive Control of a Quadrotor Helicoptor," *Elsevier Mechatronics,* 2011.

[3] S. Ge, C. Hang, T. Lee and T. Zhang, Stable Adaptive Neural Network Control, Norwell: Kluwer Academic Publishers, 2002.

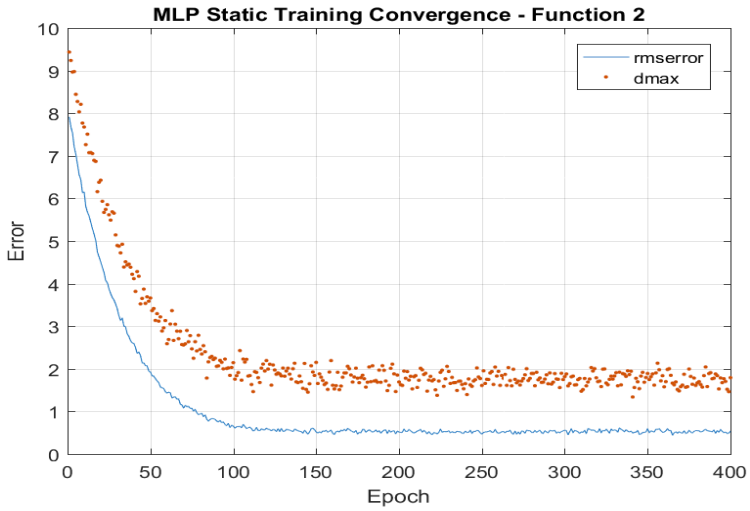
# APPENDIX I

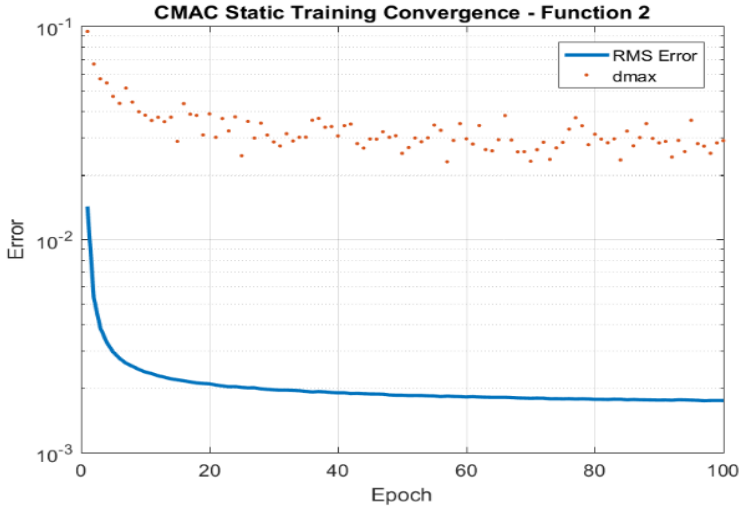
## Comparison of MLP and CMAC Static (Offline) Training Error Convergence

Nonlinear z Term

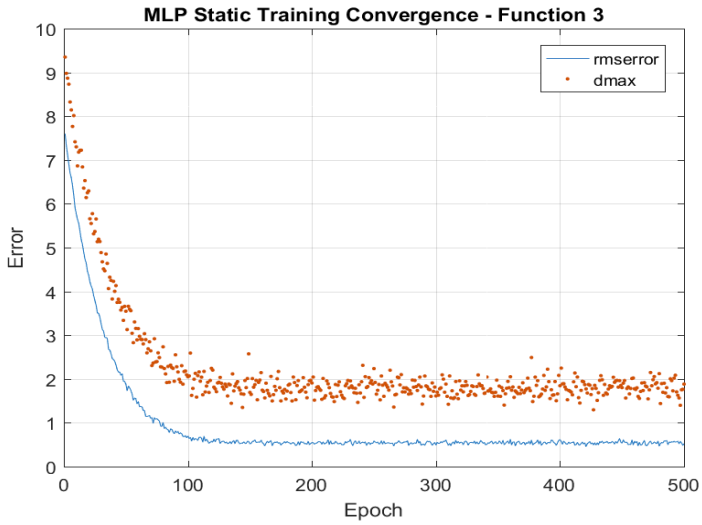



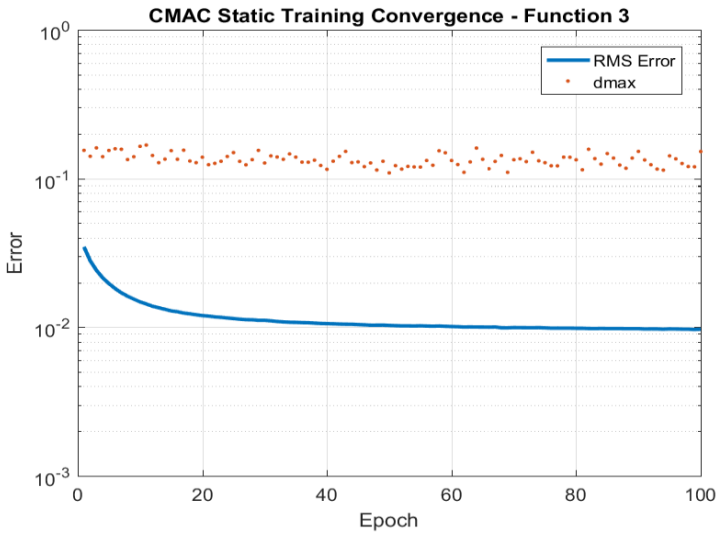

Nonlinear ϕ Term



Nonlinear θ Term




Nonlinear ψ Term